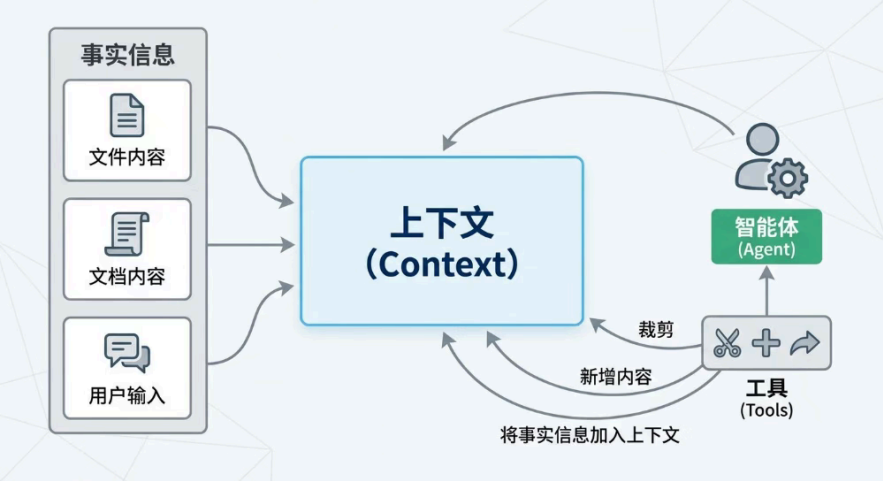
Agentic Coding场景下基于职责分离的上下文管理思路分享

原创 物流解决方案&电 大淘宝技术 2026年1月28日 00:38 浙江 570人

本文提出了一种在 **Agentic Coding 场景下基于“职责分离”思想的上下文管理新思路**：将工具调用解耦为 **“行为”（如 open_file）和“影响”（如 IDE 中实时更新的文件内容）**，通过结构化、模块化（如 <ide> 块）、动态组装的上下文设计，替代传统将大量原始数据（如完整文件内容）直接塞入上下文的做法；同时引入“行为–影响分离”“记忆/遗忘机制”“事实与行为记忆区分”“延迟卸载”等策略，系统性缓解长上下文导致的注意力稀释、信息过载、内容过期与性能退化等问题，提升 Agent 在复杂编码任务中的稳定性、可维护性与上下文利用效率。该思路虽源于 coding 场景，但具备跨任务复用潜力。

01 引言



本文主要是S1尝试进行模型为主的自动化编码过程中，遇到了长任务下上下文的爆炸以及短任务中注意力的分散问题，尝试了模型压缩、FIFO、多agent上下文隔离等各种策略后，在摸索过程中逐渐演变出的的一个设计思路，核心思想是工具的职责分离。

下文会基于语言模型的原理、特点以及在实践过程中遇到的问题，从工程中职责分离的角度出发，尝试一种新的上下文管理的思路，通过将工具分为行为和影响，通过结构化上下文的动态

管理来尝试解决上下文中的各类问题，虽然是在coding场景下的尝试，但思路也许也可以被复用到其他类型的场景中。

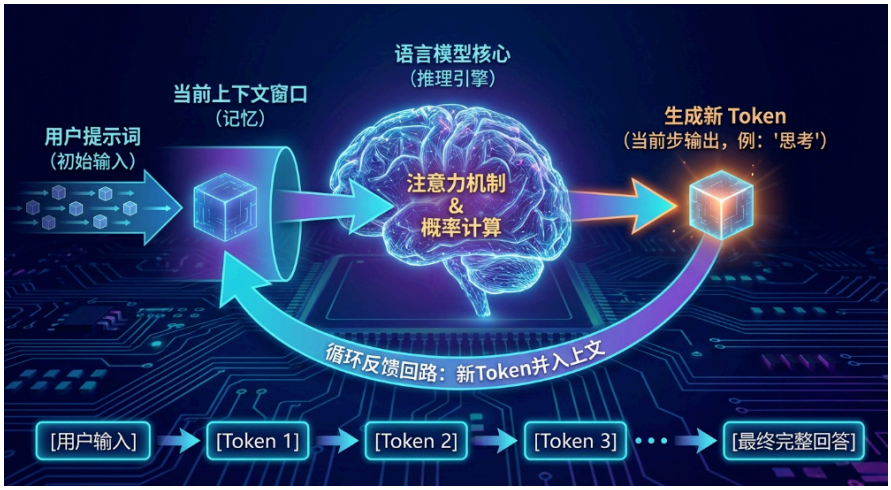
02 什么是上下文

大型语言模型（LLM）本质上是一个基于已有 token 序列预测下一个 token 的概率系统。当接收到一段文本时，模型将其转换为 token 序列，通过神经网络计算后输出下一个 token 的概率分布，并将该 token 追加到序列末尾，循环往复，直至生成结束符。

在一次请求中，“我们”输入给模型的完整内容即为上下文（Context）。简单来说，上下文就是模型“看到”的、用于理解当前语义或预测下一个词的所有前置信息。
在一个典型的 Agent 应用中，上下文通常包含以下四类内容：

1. 用户输入（User Input）
2. 工具调用与返回（Tool Calls & Responses）
3. 历史对话（Conversation History）
4. 工具定义（包括 MCP 等元信息）

对用户而言，可控部分仅限于“用户输入”；而对 Agent 开发者来说，其余三部分才是可设计、可优化的核心——这正是上下文工程（Context Engineering） 的核心任务：**在正确的时间，以正确的结构，向模型传递正确的信息，使其做出最恰当的响应或决策。**



token的多次生成循环是发生在模型内部的，在这次请求中“我们”输入的**完整内容**就是上下文。简单来说，上下文就是模型“看到”的、用于帮助它预测下一个词或理解当前语义的前面的一段文字。
在一个典型的agent应用中，上下文由下面的部分组成：

1. 用户输入

2. 工具调用
3. 历史对话
4. 工具的定义（包括mcp）

对用户来说能够掌握的部分只有“用户输入”，对于agent的开发者来说则是除了“用户输入”以外的所有内容，上下文工程指的就是agent开发者在软件开发过程中对上下文中包含的内容的组装。

上下文工程关注的是：如何把正确的信息，在正确的时间，以正确的结构，传递给模型，使其能做出最恰当的响应或决策。

上下文工程的核心工作

活动	说明
上下文设计（Context Design）	规划上下文应包含哪些元素（如系统提示、对话历史工具输出记忆片段等）。
上下文组装（Context Assembly）	在运行时动态拼接过滤排序相关信息，形成最终输入。
上下文压缩（Context Compression）	在有限 token 窗口内，通过摘要、选择性保留、向量检索等方式精简冗余信息。
上下文缓存与管理（Context Management）	维护短期对话状态和长期记忆，支持跨轮次一致性。
上下文评估与优化（Context Optimization）	通过 A/B 测试、人工评估或自动指标，迭代改进上下文结构。

03 上下文的影响：为何“长了反而变笨”？

模型本质上是一个么得感情的计算机器，用户输入是模型被动感知环境的方式，而工具则是模型主动感知和影响环境的手段。基于语言模型的原理，我们知道模型的输出取决于输入和模型的固有参数，对应用层开发来说模型的固有参数无法调整，能控制的只有输入内容。同时我们知道模型的参数维度是固定的，因此模型能感知到的特征是有限的，当用户的输入过多时能明显感觉到模型似乎在“变笨”。

这种现象通常被称为“上下文长度增加导致性能下降”或“长上下文退化”（long-context degradation），是当前大语言模型（LLM）在处理超长输入时常见的一种局限性。具体表现为：

注意力稀释

随着输入文本变长，模型的注意力机制需要在更多 token 之间分配注意力权重。关键信息可能被大量无关或冗余内容“稀释”，导致模型难以聚焦于真正重要的部分，从而影响推理、问答或摘要等任务的准确性。

位置编码失效或失真

大多数模型使用位置编码（如 RoPE、ALiBi 或绝对位置嵌入）来感知 token 的顺序。当输入远超训练时的最大上下文长度（例如训练时最大为 4K，但推理时输入 32K tokens），位置编码可能无法准确表示远距离 token 的相对关系，造成时序混乱，进而影响理解。

信息过载与噪声干扰

输入内容越多，越可能包含重复、矛盾或无关的信息。模型缺乏有效的信息筛选机制，容易被噪声干扰，甚至“迷失”在冗长文本中，导致输出变得模糊、不相关或逻辑混乱——看起来像是“变笨了”。

训练-推理不匹配

模型在训练阶段很少见到极长的上下文样本，因此对长文本的泛化能力有限。即使通过外推技术（如 YaRN、NTK-aware scaling）扩展上下文窗口，其在长上下文下的表现仍可能显著弱于短上下文。

计算资源限制带来的近似处理

为应对长上下文带来的计算开销（如注意力复杂度为 $O(n^2)$），一些系统会采用滑动窗口、稀疏注意力或缓存截断等策略，这些近似方法可能无意中丢弃关键信息，进一步降低模型表。

04 工具的上下文



LLM 本质是 token 预测器，仅能输出文本。要使其成为生产力工具，需通过工具调用机制扩展其能力：模型输出结构化指令 → 程序解析并执行 API → 将结果作为新上下文返回。

若将 LLM 比作大脑，工具就是眼睛、手、脚；若比作人类，工具就是电脑、书本。所有输入——用户指令、工具定义、调用记录、系统提示——本质上都是上下文的一部分。

在单轮 Agent 场景中，上下文膨胀主要来自：

- 工具调用返回大量数据（如读取整个文件）
- 用户上传长文档

开发者虽无法控制用户输入，但可优化工具的上下文表达方式。工具上下文分为两类：

- 静态定义：工具功能描述（编码时确定）
- 动态交互：调用参数与返回结果（运行时生成）

本文并不讨论工具定义的方式，而是直接讨论会导致上下文快速增长的动态交互部分。

工具实现的思路和问题

在coding场景中，最经典的工具就是read_file和write_file，即文件读写工具。
在提供read_file工具时，最简单的方式就是直接读取并返回完整的文件内容，为方便起见我们叫这个工具read_full_file。

> 需要注意的是部分老项目或不规范项目中可能会存在编码不一致的问题，例如utf8和gbk混用的情况，如果未使用准确的编码读取文件内容可能导致注释、非英文常量无法被模型识别导致无法正确理解“业务含义”（代码逻辑之外的信息）。在java项目中推荐这两个库juniversalchardet 和 cpdetector 来通过部分字节推测文件编码以避免此类问题。
> 当然最佳实践是通过改造来完成项目编码的统一，最佳实践是统一使用utf-8编码。

```
String readFullFile(String path){
  //这里去读取文件内容，然后return出去
}
```

> ⚠注意：工具的返回值也是上下文或者说提示词的一部分

read_full_file有一个显而易见的问题：

- “完整的文件可能很大”导致上下文快速膨胀。

此外还有一些和模型的特点相关的问题：

- 在找不到路径时需要明确告知模型 "xx路径不存在"，而不能告知模型“”，否则模型可能会陷入工具调用的无限循环（qwen系列比较明显）。
- 模型输入的path不一定准确，可能输入的是类的限定名、文件名；或是缺失部分路径、缺失文件后缀等。

对于path不准确的问题，可以基于一个简单的思路来进行处理，即先搜索在读，通过关键词匹配对path进行搜索，如果能定位到唯一的文件则读取此文件，否则明确告知模型现在有多个相

似的文件，需要模型重新再生成一次工具调用。
在read工具和write工具混用的情况下，read_full_file还会出现一个新的问题：

> 我们先不关注write工具的具体实现，只需要知道write后文件内容肯定发生了变化

- 同一份文件内容在编辑后已经失效了，需要重新读取文件内容，而历史工具调用的respone和新的response存在大量重复，如果是多次编辑则会导致上下文快速膨胀。

这是因为我们将文件内容放在工具调用的返回值中，发送给模型的消息类似于下面的格式：

```
tool_call:read_full_file filePath
tool_response:filePath.content //第一次读取
tool_call:write_file filePath xxxx
tool_response: 更新成功
tool_call: read_full_file filePath
tool_response:filePath.content //第二次读取
```

为什么模型需要读取多次或者说为什么我们希望模型读取多次?
在使用ai coding工具时，我们有时会看到模型输出“让我看看是否已经完成了所有变更”，因为模型在进行write时能够感知到的只是进行了这个行为，而并没有看到这个行为的“影响”。

> 在coding场景中，“影响”指的是修改后代码的内容。

为了解决read_full_file 工具的长上下文问题，可以引入按块读取工具read_block_file，即模型在调用工具时可以选择读取的行的数量，在理想情况下，模型可以只读取部分内容：

```
String readBlockFile(String path,int startLine,int lineCount){
  //这里去读取文件内容，然后return出去
}
```

但这个工具同样也存在其他问题

- 模型并不知道需要的内容在哪，可能还是会多次调用块读取工具最后把整个文件都读取，这反而导致了和模型的更多次交互（增加了成本）。
- 模型读取块的时候可能忽略了后续代码的影响，导致思路错误。

> 输入的token也是会计费的，用块工具读取完整的文件内容比直接读取完整的文件内容会增加多次重复上下文的调用，导致成本的上升。

在最坏情况下（需要的逻辑在文件末尾）比read_full_file花费更多的成本，且上下文更多（因为还有工具调用），并且和write工具联合使用时的问题仍然存在。

为了解决write工具联合使用时的问题，我试图引入“按需卸载”的策略，即在发现write工具后判断文件内容已经变更时就去掉历史调用中的read工具的返回值，于是我将工具划分为两个类

型，即读和写，当读写在操作同一个实体（这里指文件）时，如果有写操作则卸载掉上次的读操作，但这导致模型的幻觉现象变得更加严重，深入分析了模型的输出后，我发现并不能实时卸载，应该延迟一段“时间”（一些读写操作）后再卸载，否则模型很难感知到上次的行为，容易出现重复操作。简单来说就是保持最近的几次操作，例如50次。

> 在aone copilot中使用了类似的思路，大约只会保留最近250次的操作内容，可以右键打开devtools来查看交互的对话内容。

但是这个思路只能解决read、write文件的长上下文问题，难以推广到grep、getDependency等工具上，并且在工程上实现不够优雅。
暂且记下这些问题，回过头来先看write工具，write工具至少需要2种

1. 块替换
2. 完整覆写

```
String replace(String path,String search,String replace){
  //找到search块并替换为replace
}
String write(String path,String content){
  //直接覆盖path的内容，如果不存在则创建一个文件
}
```

write工具的主要问题是工具调用内容较多，相比read工具，write只需要确保内部替换逻辑是准确的，且在出现问题时能够准确反馈问题可以让模型重新推理即可，但在实际测试中和模型及框架有关的两个点需要注意：

1. 模型几乎不会产出\r符号，因此如果项目是在windows下操作的需要注意\r符号。
2. 模型在输出\t时容易多一次转义，例如参数需要为\t，但模型输出\\t（这应该是通过json格式约束模型输出工具调用导致的问题）。

write工具中也有path参数，因此也需要类似read工具的**路径推测**能力来降低模型进行工具调用失败的概率。
其他的**读工具**都可以参考read_file的问题，即信息过期、冗余等问题。

1. 完整读 的缺陷：
   上下文爆炸：大文件直接塞入上下文。
   路径模糊：模型可能传入类名、缺后缀等，需支持模糊匹配。
   内容过期：write 后未刷新 read 内容，导致历史响应冗余。
2. 部分读 的局限：
   模型不知关键代码位置，可能多次调用，反而增加成本。
   与 write 联用时仍存在重复内容问题。
   最坏情况下（需读末尾）比全读更耗 token。

3. “按需卸载”策略的陷阱：

   早期尝试在 write 后立即移除旧 read 响应，结果加剧幻觉——模型因缺乏行为反馈而重复操作。实践表明，应延迟卸载（例如保留最近 50–250 次操作），让模型充分感知行为影响。



回过头来思考人在coding场景下的行为，当人在进行编码时，一直在观察ide中打开的代码文件，通过键盘对代码文件进行编辑（光标位置），当需要切换编辑的位置时，将光标移动到新的位置，随后开始一段新的输入。

通过拟态来优化工具的设计思路，当人在进行编码时一直在观察的是IDE的界面，同时通过鼠标键盘在和这个界面中的元素进行交互，屏幕是有限的，所以我们观察到的内容也是有限的，这和我们期望的agent的上下文是“固定”的非常类似。

当模型试图进行一次工具调用时，本质上是为了“观察”或“影响”环境，也可以说是为了“读”和 “写”环境，这是模型主导的“行为”，环境则是agent的具体业务需要的信息，例如在coding场景中可以把代码文件、文档等视为“环境”。当工具调用结束时，将工具返回的内容组装到上下文中就实现了让模型观察环境或是了解造成的影响。

在现有思路下工具包含了两个职责

1. 行为
2. 影响的反馈

以read_file为例，实际上这个工具的目的是“获取文件内容”并发送给模型，行为是获取指定文件的内容，影响是在上下文中增加文本的内容，最终发送给模型，使得模型看起来像是阅读了代码似的。

而当人在阅读文件时，实际上是进行了一次“打开文件”的行为，然后造成了IDE现在显示我们“打开文件”的内容，所以我们（语言模型）可以通过“看”屏幕来阅读文件内容。

基于这个思路，我在system prompt中增加了一个固定存在但内容“实时”更新的块，这个块存储模型看到的事实，同时不再有读文件的工具，而是变成了open_file和close_file两个操作，当模型调用open_file工具时，是进行了一次“行为”，而工具的影响是“行为的结果”以及“IDE中显示文件内容”。

> 实时更新是指每次发送给模型前进行更新

```
String openFile(String path){
```

```
    //如果成功在ide中记录path被打开，返回已经成功打开文件
    //如果失败原因是什么
}
```

每次组装发送给模型的内容时，重新生成IDE的内容，IDE中包括“文件内容”。
参考这个思路，我们可以将“目录”、“搜索”、“引用关系”、“变更清单”、“TODOLIST”等工具都视为IDE中的模块，并提供“行为工具”来调整这些块的内容。
同时甚至可以将这些块的展示/隐藏也全权交给模型决策，即由模型自主管理。
例如

1. 我们现在打开的文件较多，可以在上下文中增加“警告信息”，让模型将不重要的文件进行关闭。
2. 打开的某个文件较大，让模型聚焦到某个块
3. 打开的目录已经使用完毕了（搜索过程），模型自主的关闭目录
4. 打开的文件已经不需要再使用了，模型自主的关闭此文件
5. 等等

但人和模型还是有本质上的区别，抛开逻辑能力的差异，最重要的是“记忆”，人可以“记住”刚刚打开的文件，即使现在关闭了，大概也能记得内容并基于“记忆”中的内容来对当前打开的文件进行操作，而LM模型我们知道它是没有记忆的，之所以有“记忆”的假象，只是因为在上下文中包含了完整的信息。
为了解决这个问题，我们可以给ide一个专门的“记忆”模块。



说到记忆，我们的根本目的是让模型“记住”那些“需要”的信息，并且基于这些信息给出准确的“判断”，最佳方式是把所有的“行为”和“影响”都放在上下文中，只要模型足够“聪明”就一定能做出最准确的判断，但我们知道语言模型的上下文是有限的，且基于语言模型的原理我们知道上下文中的内容不能过多，否则会导致模型“性能”下降。

当人在阅读了文件后，可以直接关闭或切换到其他文件进行后续工作，这是因为人记住了文件中的“关键”内容而非记住了完整的文件内容。

我们则需要手动在上下文中增加“关键信息”以便让模型“记住”关键信息，在上下文充足的情况下，首选保留完整的文件内容，随着模型的逐步深入，上下文也开始捉襟见肘了，此时在系统提示词的引导下，模型会开始试图关闭或工程上主动出发关闭文件时，需要对即将关闭的文件进行一次关键信息提取，然后将关键信息记录到一个较小的备忘录中，以实现让模型仍然记得

刚刚关闭的文件中的关键信息，我们仍然可以借助一个subagent来进行关键信息提取，而如何判断文件的关键信息则由主agent给出，当主agent打开一个文件时，需要同时给出此操作的目的，让subagent基于“目的”进行关键信息提取。

随着探索的继续深入，备忘录的大小也到达了上限，我们需要从上下文中真正的删除内容，最佳方式仍然是让模型自主管理哪些部分需要被移除，哪些部分需要保留，但也可以使用简单的先进先出或者策略。

同时因为行为和影响的分离，我们可以针对每个部分的特点来单独设计压缩、裁剪、合并等行为，更加高效的利用上下文来释放模型的能力。

事实和行为

在“记忆”中有一部分非常特殊的内容，即工具调用的记录，这是告知模型“已经做过”的内 容。但有些模型的训练过程中会专门对工具调用进行训练（支持functioncalling的模型），因而导致了一些特殊的规则，例如：

1. function call 和function response需要成对出现
2. unction call 和function response需要使用特殊格式传递给模型

例如下面一个询问天气的简单示例

```
user:今天天气怎么样
assistant:function_call getDate
agent:function_response 2025-12-22
assistant:function_call getWether 2025-12-22
agent:function_response 晴天
assistant:今天是2025-12-22，天气晴
```

在一些instruct模型中，如果不使用这种形式，模型可能会出现无限循环调用方法。
因此在agent应用中，需要将“记忆”划分为两部分：事实记忆和行为记忆。
行为记忆使用function call/response 的形式在上下文中体现，而事实记忆则通过诸如打开的文件内容、操作过的文件列表、diff内容、备忘录、todo等形式。
通过上述方式组织上下文可以使得上下文保持在一定大小内，既能够充分利用上下文窗口，也可以避免上下文快速膨胀导致的性能快速下降。

传播和继承

按照上述思路，上下文被划分成一些固定的块，例如diff、路径、备忘录、todo等，我们可以通过xml将这些块进行区分，组织成一个结构化的上下文。

```
<ide>
<open_file>文件路径</open_file>
<todo>待办</todo>
<note_book>备忘录</note_book>
</ide>

```

> ⚠在使用chat和模型进行交互时，需要将ide内容放入system prompt，否则容易被模型遗忘。

在上文的思路下模型的行为和影响被拆分，且每个部分也被自然的拆分，因此我们可以针对每个任务的特点来组装上下文信息，充分利用有限的上下文。而“影响”天然就是上一次工作的结果和过程的直观展现，新的工作可以基于上一轮的“影响”继续进行。

在面向multi agent或是sub agent时，“影响”被继承，并转向另一个维度的工作，例如deep search收集知识的过程中可以被“评估者”直接评估知识是否收集完全、代码生成时则可以直接利用deep search收集的知识进行工作。在多轮对话中“行为”和“影响”均可以被继承，模型可以无缝的接受用户新的输入后开始新的工作。

07 结语

在我看来上下文工程是一个复杂且不断发展的大模型下的工程产物，本质上是由于语言模型能力的不足而出现的一种工程上的妥协，随着语言模型能力的增强，或许在未来的某一天我们可以直接依赖模型的能力而不再需要复杂的上下文工程，但是以工程应用的视角来看，资源的高效利用、性能的极致优化永远是工程师的追求。

08 团队介绍

本文作者时兮，来自淘天集团-物流解决方案&电子面单团队。一支专注于商家寄件与电子面单业务研发的技术团队。依托多元业务场景，我们持续探索与迭代技术能力，长期投入稳定性建设，为数百万商家提供稳定可靠的寄件与物流体验，为数亿包裹安全高效流转保驾护航。

¤ 拓展阅读 ¤

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 6429

留言 1

写留言



无妄 北京 2月3日

模型本质上是一个么得感情的计算机器，用户输入是模型被动感知环境的方式，而工具则是模型主动感知和影响环境的手段。

首评